Agent Mesh SRE — Blueprint & Architecture v4.0

Document version: 4.0 | Application: surajcsibm fork | July 2026

1. Executive Summary

Agent Mesh SRE v3.0 extends the v2.1 client-side simulation with a full autonomous monitoring layer. The Monitor agent now runs a continuous 30-second polling loop that injects realistic Kafka failure conditions, detects them via real or simulated signals, and fires all 11 failover scenarios without any human button press — except for the 4 approval-gated scenarios where a human must approve before the Monitor acts.

The primary architectural additions in v3.0:

- Autonomous Monitor Polling Loop (monitor-poll.ts) — 30s interval, 10-sample sliding window, evaluates all 11 trigger conditions
- Anomaly Simulation Engine (anomaly-sim.ts) — background injector cycling through all 11 scenarios every 90–150s
- KafkaJS Admin Client (kafka-admin.ts) — ISR state, consumer group lag, controller epoch from live Kafka
- Aiven REST API extensions — disk metrics, topic sizes, consumer group lag from partition responses
- Signal bridge (/api/mesh/inject-signal) — syncs client-side scenario triggers to server broker state
- Visual detection rings on all 11 scenario buttons — cyan detected, amber auto-fired, purple suppressed
- Autonomous topic healing — affected topics auto-heal 90s after each scenario

2. Updated System Overview

```
Browser (Next.js App Router)
│
├── Autonomous Monitor Layer (server-side)
│   ├── anomaly-sim.ts     — injects broker anomalies + emits auto-trigger-scenario SSE
│   ├── monitor-poll.ts    — 30s poll loop, evaluates 11 trigger conditions
│   ├── kafka-admin.ts     — KafkaJS admin: ISR, consumer lag, controller epoch
│   └── aiven-admin.ts     — Aiven REST: disk metrics, topic sizes, CG lag
│
├── Signal Bridge
│   ├── /api/mesh/inject-signal  — writes broker state from client trigger
│   └── /api/mesh/poll           — returns poll loop state + latest snapshot
│
└── Client Simulation (unchanged from v2.1)
    ├── useMeshStream.ts  — handles auto-trigger-scenario + auto-topic-heal SSE
```

```
    ├── client-sim.ts     — runs full MRAL animation for all 11 scenarios
    └── Dashboard.tsx     — detection rings, TopicsPanel monitor alerts
```

3. New & Modified Files

3.1 New Files (v3.0)

File	Purpose
src/lib/kafka-admin.ts	KafkaJS admin singleton — ISR, consumer lag, controller epoch. Falls back to zero struct in MOCK mode.
src/lib/monitor-poll.ts	30s autonomous polling loop. 10-sample ring buffer. All 11 trigger condition evaluators. Cooldown + dedup.
src/lib/anomaly-sim.ts	Background anomaly injector. Cycles all 11 scenarios every 90–150s. Emits auto-trigger-scenario SSE.
src/app/api/mesh/poll/route.ts	GET returns PollLoopState + latestSnapshot. POST start/stop the loop.
src/app/api/mesh/inject-signal/route.ts	POST updates live broker state via patchBrokerState. Called by handleTrigger fire-and-forget.

3.2 Modified Files (v3.0)

File	Change
src/lib/mesh.ts	Added patchBrokerState() export. Changed scenarioRunning: boolean to activeScenarios: Set<string> for concurrent scenario support.
src/lib/aiven-admin.ts	Added getServiceMetrics(), getTopicSizeAndConsumerGroups(), getClusterTopicSizes().
src/lib/types.ts	Added BusEvent types: auto-trigger-scenario, auto-topic-heal, approval-new, approval-update.
src/components/useMeshStream.ts	Added SSE handlers for approval-new, approval-update, auto-trigger-scenario, auto-topic-heal.
src/components/Dashboard.tsx	Detection rings on all 11 scenario buttons. MonitorDetection hook. TopicsPanel monDet prop. Monitor alert chips on topic rows.
src/components/panels/TopBar.tsx	Added MonitorPollPill — pulsing cyan dot with cycle count and detected state.
src/components/panels/ScenarioPanel.tsx	Added MonitorStatusCard at top of scenario list. Shows poll running state and last detected scenarios.
src/app/api/mesh/stream/route.ts	startMonitorPolling() + startAnomalySimulation() called on SSE connect.

4. Autonomous Monitor Architecture

4.1 Signal Sources

Source	API	Data Collected
Consumer Group	KafkaJS admin.fetchOffsets() + describeGroups()	lag, memberCount, state — payments-consumer, share-group-1
Cluster API	Aiven REST GET /service/{s}/stats	controllerEpoch, brokersOnline, underReplicatedPartitions
Topic Metrics	Aiven REST GET /topic/{name}	partition sizes, consumer_groups lag per partition
Disk Metrics	Aiven REST POST /metrics {period: hour}	diskUsedPercent (null on free-tier plans — falls back to topic size sum)
Broker Simulation	getMeshState().broker (always read)	Simulation state baseline — always merged before real signals

4.2 All 11 Trigger Conditions

Scenario	Gate	Primary Signal	Threshold
Consumer Lag Spike	Approval	consumerGroups.payments-consumer.lag	> 10,000 AND growing 3 consecutive samples
KRaft Controller Failover	Auto	broker.controllerEpoch	epoch increases beyond last cached value
Share Group Rebalance	Approval	consumerGroups.share-group-1.lag	> 10,000 AND growing
False-Positive Suppression	Suppress	rebalanceState + lag	Rebalancing but lag < 1,000 — suppress action
Schema Registry Mismatch	Approval	signalledScenarios.schema-mismatch	Signal within 90s window
Broker Disk Saturation	Auto	diskUsedPercent OR topic size sum	> 80% OR topic bytes > 500 MB
Under-Replicated Partitions	Approval	underReplicatedPartitions	> 0 sustained 2+ samples
Producer Timeout Storm	Auto	signalledScenarios.producer-timeout	Signal within 90s window
Consumer Session Timeout	Auto	consumerGroups state = dead	memberCount = 0 OR state = Dead
Log Compaction Lag	Auto	signalledScenarios.compaction-lag	Signal within 90s window
Partition Imbalance	Suppress	broker count change	Detected but no action — benign pattern

4.3 Autonomous Trigger Flow

```
t=0s:   App loads → SSE connects → startMonitorPolling() + startAnomalySimulation()
t=35s:  AnomalySim: first anomaly injected into broker state
t=35s:  AnomalySim: emits auto-trigger-scenario SSE → useMeshStream calls runClientScenario()
t=35s:  Full MRAL animation plays on canvas
        → approval-gated: Policy Gate modal appears, user approves/rejects
        → auto-execute: scenario fires with no human input
        → suppress: Monitor detects, logs 'suppressed', takes no action
t=35s:  runClientScenario calls POST /api/notify → email sent if NOTIFICATION_EMAIL set
t=65s:  Poll loop detects broker state change → audit log [POLL] entry
t=125s: AnomalySim heal timer fires → broker state restored
t=125s: auto-topic-heal SSE emitted → runTopicHeal() triggered for affected topics
t=150s: Next anomaly injected (90–150s random interval)
```

5. New API Routes

Route	Method	Auth	Description
/api/mesh/poll	GET	Required	Returns PollLoopState (cycleCount, running, detectedThisCycle, cooldowns) + latestSnapshot
/api/mesh/poll	POST	Required	start or stop the Monitor polling loop (body: { action: 'start' | 'stop' })
/api/mesh/inject-signal	POST	Required	Writes anomaly values to live broker state via patchBrokerState(). Called by handleTrigger.

6. Updated Environment Variables

Variable	Required	Description
KAFKA_MODE	Yes	real or mock (lowercase). Controls whether KafkaJS/Aiven connections are made.
KAFKA_BOOTSTRAP	REAL only	Bootstrap host:port, read by kafka-admin-cfk.ts. Must match KAFKA_BOOTSTRAP_SERVERS exactly.
KAFKA_BOOTSTRAP_SERVERS	REAL only	Bootstrap host:port, read by kafka.ts. Two separate env vars for the same setting — a known outage trap if they diverge.
KAFKA_SSL_ENABLED	REAL only	Must be exactly "true" or "false". Any other value in real mode, including an empty string, now throws at client

		construction rather than silently defaulting to TLS.
KUBECONFIG_BASE64	REAL only	Base64-encoded kubeconfig for the least-privilege agent-mesh-vercel ServiceAccount. Never the full admin kubeconfig.
KAFKA_CA_CERT_BASE64 (retired)	Retired	Not used on DOKS — the external listener is unauthenticated PLAINTEXT with no CA cert. Was required only for the earlier Aiven/Civo setup.
AIVEN_TOKEN (retired)	REAL only	Not used — the Aiven REST API is no longer part of the stack since the DOKS migration.
AIVEN_PROJECT (retired)	REAL only	Not used — see AIVEN_TOKEN above.
AIVEN_SERVICE (retired)	REAL only	Not used — see AIVEN_TOKEN above.
NOTIFICATION_EMAIL	Optional	Recipient email for autonomous scenario notifications
SMTP_HOST / SMTP_USER / SMTP_PASS	Optional	SMTP credentials for email notifications
GOOGLE_CLIENT_ID / SECRET	Yes	Google OAuth credentials
NEXTAUTH_SECRET / URL	Yes	NextAuth JWT signing secret and deployment URL

7. Detection Ring Visual System

Ring State	Colour	Badge	Meaning
Conditions detected, within cooldown	Cyan #22d3ee	● detected	Monitor sees anomaly, trigger blocked by 5-min cooldown
Autonomously fired (auto/approval gate)	Amber #fbbf24 glow	⚡ auto-fired	Monitor triggered full MRAL — no button press
Intelligently suppressed	Purple #a78bfa	⊘ suppressed	Monitor detected but decided not to act (scenarios 04, 11)
No detection	None	—	Broker state healthy, no conditions met

Rings appear within 30 seconds of anomaly injection and persist for 2 minutes via the recentDetections buffer in monitor-poll.ts.

8. Topic Auto-Healing

Each scenario in anomaly-sim.ts declares an affectedTopics array. After a 90-second heal delay, the simulator emits auto-topic-heal SSE events for each affected topic. The useMeshStream handler calls runTopicHeal(), which runs a dedicated MRAL healing cycle visible on canvas.

Scenario	Topic Healed	Status Before Heal
lag-spike	ops.kafka.metrics.v1	critical (lag 18,500)
share-group	ops.requests.v1	degraded (lag 8,500)
schema-mismatch	ops.actions.audit.v1	degraded (lag 7,800)
disk-saturation	ops.lessons.v1	degraded (lag 3,200)
under-replication	ops.incidents.v1	degraded (lag 5,100)
producer-timeout	ops.kafka.metrics.v1	degraded (lag 4,900)
consumer-session-timeout	ops.kafka.metrics.v1	degraded (lag 3,800)
compaction-lag	ops.lessons.v1	degraded (lag 2,400)

9. Approval Gate Architecture

9.1 Manual Triggers (unchanged)

Client button click → runClientScenario() → client-sim dispatch → _pendingApprovalCallback → modal shows → user clicks → resolvePendingApproval() + POST /api/mesh/approve

9.2 Autonomous Triggers (new in v3.0)

AnomalySim emits auto-trigger-scenario SSE → useMeshStream calls runClientScenario() — same client-side path as manual triggers. The _pendingApprovalCallback mechanism handles approval gates natively. No separate server-side approval flow needed.

The 4 approval-gated scenarios:

- Consumer Lag Spike — kafka.scaleConsumers
- Share Group Rebalance — kafka.acknowledgeShareGroupRebalance (acknowledge-only; no client-side mutation)
- Schema Registry Mismatch — kafka.updateSchemaCompatibility
- Under-Replicated Partitions — partition reassignment

10. Extension Points

Adding a new autonomous scenario

- Add to EXTRA_SCENARIOS in Dashboard.tsx
- Add to SCENARIO_AUTO_TOPICS in Dashboard.tsx
- Add case in client-sim.ts runClientScenario switch

- Add Anomaly entry in anomaly-sim.ts ANOMALIES array with inject(), heal(), affectedTopics
- Add trigger evaluator function in monitor-poll.ts EVALUATORS array

Adjusting detection sensitivity

Edit threshold values in monitor-poll.ts evaluator functions. Key constants: POLL_MS (30,000ms), WINDOW_SIZE (10 samples = 5 min history), COOLDOWN_MS (300,000ms = 5 min). In anomaly-sim.ts: MIN_INTERVAL (90,000ms), MAX_INTERVAL (150,000ms), HEAL_DELAY (90,000ms).

11. v4.0 — Infrastructure Migration & Bug Fixes (July 2026)

This section documents changes made during a single extended infrastructure session, superseding no prior content — all v3.0 client-side simulation, monitor-poll, and anomaly-sim architecture described above remains accurate and unchanged. What changed is the underlying Kafka infrastructure the app connects to in Real Cluster mode, plus five bugs found and fixed in the Kafka client code along the way.

11.1 Infrastructure Migration

The live demo cluster moved from Confluent for Kubernetes on Civo Cloud to Confluent for Kubernetes on DigitalOcean Kubernetes Service (DOKS), in KRaft mode, 3 brokers + 3 controllers + 1 schema registry, namespace confluent, cluster name agent-mesh-sre, region nyc1.

- Root cause for the migration: the Civo cluster developed a node/network-level fault (DNS and Civo API egress from pods failing with “Operation not permitted” — an active block, not a timeout) outside the scope of Kubernetes object-level fixes. Rather than block Bobathon prep on a third-party infra fix, a parallel cluster was stood up.
- External access uses type: nodePort rather than type: loadBalancer. DigitalOcean provisions one LoadBalancer per Service — with CFK's per-broker addressing this meant four separate LBs (~$48/month) instead of one. NodePort exposes the chosen ports on every node via kube-proxy, so a single node IP works as the shared host regardless of which node a broker pod lands on, at no additional cost.
- All CFK CRD field names (dataVolumeCapacity, storageClass.name, dependencies.kRaftController.clusterRef.name, dependencies.kafka.bootstrapEndpoint) were confirmed directly via kubectl explain against the live cluster rather than assumed, avoiding a repeat of an earlier costly guessing loop on Civo.
- Current external address: 64.227.25.232:31090 (bootstrap), :31091–31093 (brokers 0–2). This is a short-lived demo cluster — re-verify with kubectl get nodes -o wide if it has been recreated.

11.2 Kafka Client Bugs Found and Fixed

Six distinct issues were found in the Kafka-connecting code, five together and a sixth in a later session, while debugging why the deployed app could not connect properly even after the cluster itself was confirmed healthy and externally reachable.

- kafka.ts checked KAFKA_MODE for exact uppercase "REAL", contradicting the documented lowercase convention and an already-fixed identical bug in runtime-mode.ts. Fixed to case-insensitive comparison.
- kafka.ts and kafka-admin.ts always included a sasl block in the KafkaJS config with username/password forced non-null via TypeScript's ! operator, which broke connections to the unauthenticated demo cluster. Fixed by making the sasl key conditional on credentials actually being present.
- kafka-admin-cfk.ts cached its Admin client in a module-level singleton, which raced with itself under Vercel's serverless model — one invocation's disconnect() could tear down a socket a concurrent invocation on the same warm instance was still using, producing “write after end” errors. Fixed by building and disposing a fresh client per call.
- kafka.ts reads KAFKA_BOOTSTRAP_SERVERS while kafka-admin-cfk.ts reads KAFKA_BOOTSTRAP — two different env var names for the same setting. Vercel had only the latter set, and to a stale address from the already-decommissioned Civo cluster, producing silent connections to dead infrastructure from one code path while the other worked correctly. Both variables must be set identically; consolidating onto one name is a recommended follow-up, not yet done.
- The real Kafka admin poll (describeCluster, topic metadata) lived inside the code path triggered by the /api/mesh/stream SSE connection, which is hard-capped at 60 seconds on Vercel's Hobby tier, combined with a setInterval loop set up at module scope that does not reliably persist across serverless cold starts. Real broker metrics rarely had a chance to complete and overwrite the simulated fallback value. Fixed by calling the poll cycle directly inside GET /api/mesh/poll, a short, frequently-hit route with no timeout risk.
- (Found in a later session.) kafka.ts's SSL flag used sslEnabled = process.env.KAFKA_SSL_ENABLED?.toLowerCase() !== "false", so an absent or blank value defaulted to TLS-on rather than TLS-off. KAFKA_SSL_ENABLED was blanked to an empty string during a Vercel env edit, which aimed a TLS handshake at the plaintext listener and broke every Kafka-touching route in production. Fixed with an explicit guard that throws in real mode on any value other than exactly "true" or "false", with the raw value in the error message.

11.3 Known Issue — Not Fixed This Session

The dashboard's Broker card and Kafka Topics sidebar read from lib/client-sim.ts, which was never wired to consume the real /api/mesh/poll snapshot. This is why the UI displays “Brokers Online: 1” and placeholder topic names even when the backend genuinely reports 3 healthy brokers and the correct 7 topics — confirmed via direct inspection of the /api/mesh/poll JSON response, bypassing both the dashboard UI and Vercel's log viewer. Deliberately left unfixed given proximity to the Bobathon deadline and confirmed low risk: scenario playback (MRAL loop, tool calls, lessons, notifications) is unaffected and runs correctly end-to-end against the real cluster.

Recommended fix for a future session: merge the real snapshot into the same state object client-sim.ts feeds to Dashboard.tsx / AgentCanvas.tsx.